



Volume

Problem set



فایل‌های **WAV**، نوع رایج از فایل‌های صوتی هستند. این فایل‌ها، صدا را به عنوان دنباله‌ای از "نمونه‌ها" ذخیره می‌کنند: اعدادی که مقدار سیگنال صدا در یک زمان مشخص را نشان می‌دهند. فایل‌های WAV با یک header به طول 44 بایت شروع می‌شوند که شامل اطلاعاتی راجع به خود فایل است: اندازه‌ی فایل، تعداد نمونه‌ها بر ثانیه و اندازه‌ی هر نمونه. بعد از header، فایل WAV شامل یک دنباله از نمونه‌ها می‌شود که هر نمونه یک عدد صحیح 2 بایتی (یا 16 بیت) است که سیگنال صدا را در یک زمان مشخص نشان می‌دهد.

ضرب کردن هر نمونه در یک ضریب دلخواه، باعث تغییر ولوم و حجم صدا می‌شود. ضرب کردن مقدار هر نمونه در 2.0، باعث 2 برابر شدن حجم صدای اولیه می‌شود، درحالی که ضرب کردن هر نمونه در 0.5، باعث نصف شدن حجم صدای اولیه می‌شود.

در فایلی به اسم **volume.c** در فولدري به اسم **volume**، برنامه‌ای بنویسید که توسط آن بتوان حجم صدای یک فایل صوتی را تنظیم کرد.

برای این مسئله، شما باید عملکرد کدی که توسط تیم CS50 در اختیار شما قرار گرفته را ارتقا دهید.

Distribution code را دانلود کنید.

در <https://cs50.dev> وارد شوید، روی ترمینال کلیک کرده و `cd` را به تنهایی اجرا کنید، ترمینال شما باید شبیه نمونه زیر باشد:

```
$
```

حالا دستور زیر را اجرا کنید تا فایل ZIP به اسم `volume.zip` را دانلود کنید:

```
wget https://cdn.cs50.net/2024/fall/psets/4/volume.zip
```

حالا با اجرا کردن دستور زیر فولدری به اسم `volume` بسازید:

```
unzip volume.zip
```

از حالا به بعد دیگر نیازی به فایل zip ندارید پس با دستور زیر به همراه Y آن را حذف کنید:

```
rm volume.zip
```

حالا دستور زیر را اجرا کنید تا به فولدر مورد نظر منتقل شوید:

```
cd volume
```

ترمینال شما باید شبیه نمونه زیر باشد:

```
volume/ $
```

اگر تمام این کارها را با موفقیت انجام دادید، باید بتوانید با دستور `ls` فایل‌ها به اسم `volume.c` را مشاهده کنید و با اجرا کردن دستور `code volume.c` باید بتوانید این فایل را باز کنید و کد خود را برای این مسئله در آن بنویسید. اگر اینگونه نشد، گام‌های بالا را بررسی کنید و ببینید می‌توانید جایی که اشتباه کرده‌اید را پیدا کنید؟

`volume.c` را طوری کامل کنید که حجم صدای یک فایل صوتی را به نسبت ضریب داده شده تغییر دهد.

- برنامه باید سه آرگومان خط فرمان قبول کند. اولی که نشان دهنده‌ی اسم فایل اصلی است، `input` می‌باشد. دومی که نشان دهنده‌ی اسم فایل جدیدی که باید ساخته شود است، `output` می‌باشد. سومی که نشان دهنده‌ی مقداری است که صدای فایل اصلی باید به آن نسبت تغییر کند، `factor` می‌باشد.
برای مثال، اگر `factor` برابر 2.0 باشد، برنامه شما باید حجم صدای فایل صوتی `input` را 2 برابر کند و صدای تولید شده را در فایل صوتی جدید `output` ذخیره کند.
- برنامه شما باید اول `header` فایل `input` را بخواند و این `header` را در فایل `output` بنویسد.
- سپس برنامه شما باید مابقی داده‌های فایل `WAV` را به صورت نمونه‌های 16 بیت (یا 2 بایت) بخواند و پس از ضرب کردن نمونه‌ها در مقدار `factor`، آنها را در فایل جدید بنویسد.
 - در اینجا می‌توانید فرض کنید فایل `WAV`، مقادیر علامت‌دار 16 بیتی را به عنوان نمونه استفاده می‌کند. درواقع این فایل‌ها می‌توانند تعداد بیت‌های مختلفی را در هر نمونه داشته باشند اما ما فرض را بر نمونه‌های 16 بیتی می‌گذاریم.
- اگر برنامه‌ی شما از `malloc` استفاده می‌کند، نباید هیچ نشستی حافظه‌ای داشته باشد.

فهم کد در volume.c

توجه کنید که `volume.c` آماده گرفتن سه آرگومان خط فرمان می باشد: `input` ، `output` و `factor`.

- تابع `main` یک عدد صحیح `argc` و یک آرایه از کارکترها (`char *`) که `argv` است را دریافت می‌کند.
- اگر `argc` که تعداد آرگومان‌های خط فرمان با احتساب خود اسم برنامه است، مساوی با 4 نبود، برنامه باید با چاپ نحوه درست ورود اطلاعات (`usage:`) و کد وضعیت 1، متوقف شود.

```
int main(int argc, char *argv[]) {  
    // Check command-line arguments  
  
    if (argc != 4)  
    {  
        printf("Usage: ./volume input.wav output.wav factor\n");  
        return 1;  
    }  
  
    // ...  
}
```

سپس، این فایل با استفاده از `fopen` دو فایل داده شده به عنوان آرگومان را باز می‌کند.

کار اصولی آن است که نتیجه فراخوانی `fopen` را بررسی کنید؛ اگر `NULL` بود، به این معنی است که فایل پیدا نشده یا قابل باز کردن نبوده است.

```
// Open files and determine scaling factor
FILE *input = fopen(argv[1], "r");

if (input == NULL)
{
    printf("Could not open file.\n");
    return 1;
}

FILE *output = fopen(argv[2], "w");

if (output == NULL)
{
    printf("Could not open file.\n");
    return 1;
}
```

بعدا این فایل‌ها با `fclose` بسته می‌شوند. هر وقت `fopen` را فراخوانی کردید باید بعدا `fclose` را نیز فراخوانی کنید.

```
// Close files fclose(input);  
fclose(output);
```

قبل از بستن فایل‌ها یادتان نرود که بخش‌های TODO را انجام دهید.

```
// TODO: Copy header from input file to output file  
// TODO: Read samples from input file and write updated data to output file
```

برای تغییر حجم صدا، شما باید ضریب مورد نظر را نیز بدانید؛ برای همین است که `volume.c` سومین آرگومان را به یک `float` تبدیل کرده است

```
float factor = atof(argv[3]);
```


Header فایل WAV را از فایل اصلی برداشته و در فایل جدید بنویسید.

اولین TODO ای که باید انجام دهید این است که header فایل WAV را از فایل اصلی (input) به فایل جدید (output) منتقل کنید. ابتدا باید راجع به چند نوع خاص از داده‌ها مطالبی یاد بگیرید.

تا اینجا چندین نوع از داده‌ها را در C دیده‌ایم، مثل: `int`، `bool`، `char`، `double`، `float` و `long`. با این حال در فایل header ای به نام `stdint.h` چندین نوع دیگر از داده‌ها استفاده شده که به ما کمک می‌کنند تا اندازه (بر حسب بیت) و علامت (علامت‌دار یا بدون علامت) یک عدد صحیح را مشخص کنیم. دو نوع از اینها می‌توانند در کار با فایل‌های WAV برای ما مفید باشند:

- نوع `uint8_t` عدد صحیح ۸ بیتی بدون علامت (یعنی غیرمنفی) را ذخیره می‌کند. بنابراین، می‌توانیم هر بایت از Header فایل WAV را به عنوان یک مقدار از نوع `uint8_t` در نظر بگیریم.
- نوع `int16_t` عدد صحیح ۱۶ بیتی علامت‌دار (یعنی مثبت یا منفی) را ذخیره می‌کند. بنابراین، می‌توانیم هر نمونه صوتی در یک فایل WAV را به عنوان یک مقدار از نوع `int16_t` در نظر بگیریم.

بهتر است یک آرایه از بایت‌ها بسازید تا `header` فایل WAV که از فایل ورودی می‌خوانید را در آن ذخیره کنید. با استفاده از `uint8_t` که یک بایت را نشان می‌دهد، می‌توانید یک آرایه `n` بایتی برای `header` بسازید:

```
uint8_t header[n];
```

به جای `n` تعداد بایت‌ها را قرار دهید. می‌توانید از `header` به عنوان آرگومان `fread` یا `fwrite` برای خواندن یا نوشتن `header` استفاده کنید.

به یاد داشته باشید که `header` فایل‌های WAV همیشه دقیقاً 44 بایت است. توجه کنید که `volume.c` یک متغیر به اسم `HEADER_SIZE` برای شما مشخص کرده که با آن، تعداد بایت‌های داخل `header` را نشان می‌دهد

کد زیر یک راهنمایی بسیار شفاف از نحوه انجام این TODO می‌باشد:

```
// Copy header from input file to output file
uint8_t header[HEADER_SIZE];

fread(header, HEADER_SIZE, 1, input);

fwrite(header, HEADER_SIZE, 1, output);
```

داده‌های به‌روزرسانی شده را در فایل **output** بنویسید.

TODO بعدی شما این است که نمونه‌ها را از فایل **input** خوانده، آنها را به‌روزرسانی کرده و نمونه‌های به‌روزرسانی شده را در فایل **output** بنویسیم. در هنگام خواندن نمونه‌ها، می‌توانید یک حافظه موقت برای ذخیره موقت داده‌ها درست کنید. در این حافظه موقت، داده‌ها را تغییر داده و سپس محتویات این حافظه موقت را در یک فایل جدید بنویسید.

به یاد داشته باشید که می‌توانید با استفاده از نوع **int16_t**، مقدار یک نمونه در فایل **WAV** را نشان دهید. برای ذخیره یک نمونه‌ی صوتی می‌توانید یک متغیر موقت مثل زیر ایجاد کنید:

```
// Create a buffer for a single sample
int16_t buffer;
```

با وجود یک حافظه موقت برای نمونه‌ها، حال می‌توانید هر بار یک داده از فایل ورودی خوانده و درون حافظه موقت بنویسید. از **fread** برای این کار استفاده کنید. می‌توانید **&buffer**، آدرس حافظه موقت ایجاد شده، را به عنوان آرگومان ورودی به **fread** یا **fwrite** بدهید تا از این حافظه خوانده یا در آن بنویسید.

```
// Create a buffer for a single sample int16_t
buffer;

// Read single sample into buffer
fread(&buffer, sizeof(int16_t), 1, input);
```

حال برای زیاد یا کم کردن حجم صدای یک نمونه، تنها باید آن را در ضریب مشخص، ضرب کنید.

```
// Create a buffer for a single sample int16_t buffer;

// Read single sample into buffer
fread(&buffer, sizeof(int16_t), 1, input);

// Update volume of sample
buffer *= factor;
```

در آخر، باید نمونه‌ی به روزرسانی شده را در فایل `output` بنویسید.

```
// Create a buffer for a single sample int16_t buffer;

// Read single sample from input into buffer
fread(&buffer, sizeof(int16_t), 1, input);

// Update volume of sample
buffer *= factor;

// Write updated sample to new file
fwrite(&buffer, sizeof(int16_t), 1, output);
```

فقط یک مشکل وجود دارد: باید تا زمانی که هنوز نمونه‌هایی برای خواندن در فایل ورودی باقی مانده باشند، همچنان به خواندن نمونه‌ها و قرار دادن آن‌ها در حافظه موقت و به‌روزرسانی مقدار آن‌ها و سپس نوشتن نمونه‌ی به‌روز شده در فایل خروجی ادامه دهید.

- خوشبختانه مطابق مستندات، `fread` تعداد داده‌های خوانده شده را برمیگرداند. این می‌تواند در تشخیص رسیدن به انتهای فایل مفید واقع شود.
- می‌توانید `fread` را مانند مثال زیر، داخل حلقه شرطی `while` فراخوانی کنید:

```
while (fread(...)) {  
  
}
```

با اینکه این یک راهنمایی کامل است، نمونه زیر را برای حل موثر این مسئله ببینید:

```
// Create a buffer for a single sample int16_t buffer;

// Read single sample from input into buffer while there are samples left to read
while (fread(&buffer, sizeof(int16_t), 1, input) != 0)
{
    // Update volume of sample
    buffer *= factor;

    // Write updated sample to new file
    fwrite(&buffer, sizeof(int16_t), 1, output);
}
```

از آنجاییکه نسخه C ای که شما استفاده می‌کنید، مقادیر غیر صفر را به عنوان `true` و مقدار صفر را به عنوان `false` در نظر می‌گیرد، پس می‌توانید کد بالا را به شکل زیر ساده کنیم:

```
// Create a buffer for a single sample int16_t buffer;

// Read single sample from input into buffer while there are samples left to read
while (fread(&buffer, sizeof(int16_t), 1, input))
{
    // Update volume of sample
    buffer *= factor;

    // Write updated sample to new file
    fwrite(&buffer, sizeof(int16_t), 1, output);
}
```

برنامه شما باید مانند مثال‌های زیر کار کند:

```
$ ./volume input.wav output.wav 2.0
```

وقتی `output.wav` را با `control-click` کردن روی آن در مرورگر فایل، انتخاب کردن **Download** و باز کردن این فایل در `audio player` کامپیوترتان پخش کنید، صدای آن باید دو برابر صدای `input.wav` باشد!

```
$ ./volume input.wav output.wav 0.5
```

وقتی به `output.wav` گوش می‌دهید، صدای آن باید نصف صدای `input.wav` باشد!

درستی

```
check50 cs50/problems/2025/x/volume
```

استایل

```
style50 volume.c
```

نحوه ارسال

```
submit50 cs50/problems/2025/x/volume
```

CS50x Iran

Harvard's Computer Science 50x Iran

